SCREEPS OMEGA OPERATOR SERIES

Omega Developer Field Manual

Developer Field Reference
Runtime, Validation, Construction, And Private-Server Workflow

Repository: screeps_omega
Repository Version: 2751b35911042a164c0b0a3dd7438ad325f9daf8
Generated: 2026-06-18 00:18:13 PDT

PREPARED FOR PRINT REFERENCE

Omega Developer Field Manual

Repository: `screeps_omega`

Repository version used: `2751b35911042a164c0b0a3dd7438ad325f9daf8`

Generation date: `2026-06-18 00:18:13 PDT`

Source authority: `docs/user_manual.docx`

Series: Screeps Omega Operator Series

Print classification: Developer and private-server field reference

1. Orders For Use

This manual is issued for work on the active `/dev` build and the PTR private-server workflow. It describes the bot as it stands now, not a retired design target or wish list.

Before altering lower-level role code, consult the room phase, construction status, spawn queue, deterministic harness, and CPU reports. These instruments are the first line of diagnosis.

Treat `/release` as the conservative live tree. Treat `/dev` as the primary development and private-server test tree.

2. Current Theater Of Operations

- The active development target is one owned home room capable of running solo from first spawn placement through mature RCL8 construction and factory-era operation.
- Controller progress and room stability remain the principal economic objectives.
- Static defense construction is terrain-based. Exit chokes are sealed with two-tile rampart gates instead of a spawn-centered wall ring.
- CPU discipline is standing doctrine. The design target remains the real 20 CPU account cap.

3. Validation Status

- Primary validation is the deterministic harness at `scripts/validation/solo_room_harness.js`.
- The harness currently passes `bootstrap`, `foundation`, `development`, `logistics`, `specialization`, `fortification`, `command`, `bootstrap_harvest_spread`, `bootstrap_spawn_cap`, `storage_cap`, `container_usage`, and `factory_ops`.
- Live PTR validation is sufficient to confirm first-spawn bootstrap to foundation under the 20 CPU cap.
- During March 24, 2026 PTR checks, bootstrap and foundation stayed well below the 20 CPU limit. A forced high-RCL staging pass stayed roughly in the 3 to 6 CPU range while planning and running a larger workforce.
- Observer scans, power processing, nuker firing policy, and a full organic late-game solo climb remain short of full live validation.

4. Runtime Chain Of Command

- `dev/main.js` is the Screeps entry point and hands execution to `kernel_loop`.
- `dev/kernel_loop.js` handles memory hygiene, owned-room iteration, and CPU snapshot capture.
- `dev/room_manager.js` runs state collection, construction, towers, advanced structure operations, spawning, creeps, signing, directives, and HUD in a fixed order.
- `dev/room_state.js` collects shared room facts once and attaches build status and infrastructure state for downstream systems.
- `dev/utils.js` provides the shared runtime cache and movement and targeting helpers used across multiple roles and managers.

5. Room Phase Orders

PHASE	FIELD ORDER
`bootstrap`	Survive from first spawn and reach RCL2 without overcommitting the emergency workforce.
`foundation`	Place source containers, an early hub container, a controller container, and the first durable road backbone.
`development`	Fill out extensions, tower coverage, storage, and basic room defense.
`logistics`	Add the first link backbone to reduce hauling pressure.
`specialization`	Bring online terminal, extractor access, and the first lab cluster.
`fortification`	Add mature infrastructure and factory-era structure planning.
`command`	Complete observer, power spawn, nuker, and full RCL8 completion work.

6. Construction And Defense

- `construction_roadmap.js` defines the named phase goals and structure targets.
- `construction_status.js` is the shared truth for counts, readiness, and cached future-plan status.
- `construction_manager.js` places sites in roadmap order, including typed source, hub, and controller container planning, while respecting the site cap.
- `defense_layout.js` scans room exits, identifies cheaper choke lines, and seals them with walls plus two adjacent rampart gate tiles per passage.

- Future planning for links, terminal, extractor, labs, factory, observer, power spawn, and nuker is cached in room memory so the planner does not recompute heavy layout work every tick.

7. Spawn, Workforce, And Container Flow

- `spawn_manager.js` rebuilds a readable spawn queue every tick and stores it in room memory.
- The active home-room workforce remains `jrworker`, `worker`, `miner`, `hauler`, `upgrader`, `repair`, and `defender`.
- Bootstrap intentionally caps the RCL1 `jrworker` target at 2 so the room upgrades out of emergency mode instead of endlessly refilling the spawn.
- Bootstrap and fallback harvesters claim real source-adjacent harvest spots so multiple creeps do not collapse onto the same tile.
- Container roles are explicit: up to 3 source containers, 1 hub container near the anchor before storage, and 1 controller container until controller-link logistics take over.
- Workers prefer the hub buffer, upgraders prefer the controller buffer, and haulers refill those buffers before generic storage reserve work.
- The shared move helper uses the engine `moveTo` path first and only falls back to direct stepping when needed, reducing pathing fragility in live PTR checks.

8. Advanced Structure Operations

- `advanced_structure_manager.js` selects lab reactions, runs lab production, runs conservative factory production, and exposes advanced haul tasks from shared room state.
- Advanced hauling covers `lab_cleanup`, `lab_output`, `lab_input`, `factory_output`, `factory_input`, `factory_energy`, `power_spawn_power`, `power_spawn_energy`, `nuker_ghodium`, and `nuker_energy`.
- The active advanced haul task is selected from `config.ADVANCED.HAUL_TASK_PRIORITY` rather than hardcoded manager order.
- The current home-room factory policy is conservative and centered on battery production once storage energy is high enough.
- Observer, power spawn processing, and nuker fire policy still need runtime behavior beyond staged supply support.

9. Validation And CPU Drill

- Run the deterministic harness with `node scripts/validation/solo_room_harness.js`.
- Use the private-server helpers in `scripts/private_server/` for reset, upload, fast ticks, room staging, and browser access.
- Normal field sequence: start the PTR server, reset or reseed the room, upload `/dev`, run the harness, then spot-check the live room under the 20 CPU cap.
- Use `Memory.stats`, the HUD, and `ops.cpuStatus()` to confirm average CPU, bucket health, and runtime throttling state before blaming role logic.
- Prefer caches and shared state scans over repeated `room.find` calls. Memory-heavy caches are acceptable when they clearly reduce repeated CPU cost.

10. Console Command Post

`ops.help()` prints the available console commands with a short description and one-line example, keeping operator discovery in-game and current with the code.

Use `view("on")` for a quick operator dashboard, `ops.cpuStatus()` for runtime pressure, `ops.nextRCL()` for controller ETA, and `ops.hud()` or `ops.reports()` to toggle display noise independently.

11. Configuration Surfaces

- Use `config.ADVANCED` for lab policy, factory policy, advanced hauling priority, and late-structure staging targets.
- Use `config.CREEPS` for workforce targets and think intervals.
- Use `config.CONSTRUCTION` for site caps, planner cadence, and future-structure tuning.
- Use `config.DEFENSE` for choke depth and reactive defense settings.
- Use `config.LOGISTICS` for storage caps plus hub and controller container fill targets.
- Use `config.STATS`, `config.HUD`, and `config.DIRECTIVES` to control CPU visibility and operator noise.

12. Memory And Cache Ledger

- `Memory.stats` stores short CPU history, averages, and runtime mode state.
- `Memory.rooms[roomName].construction` stores planner cadence and the cached future-plan payload.
- `Memory.rooms[roomName].advancedOps` stores late-structure summary state, lab layout, preferred lab product, and advanced haul claim state.
- `Memory.rooms[roomName].defenseLayout` stores the cached exit-choke defense plan.
- `Memory.rooms[roomName].spawnQueue` stores the human-readable spawn queue snapshot.
- `Memory.runtime.rooms[roomName]` stores a lightweight live snapshot that is useful when debugging phase, creep positions, and early-room progress.

13. Known Open Fronts

- A full organic solo climb from first spawn through late RCL8 still needs more live private-server observation, even though the deterministic harness is green.
- Late-game structure fit still needs live validation in terrain-heavy rooms, especially for labs, factory, observer, power spawn, and nuker placement.
- Observer scan scheduling, power processing, and nuker fire policy remain open runtime tasks.
- Terminal balancing is still implicit rather than a dedicated late-ops subsystem.

14. Practical Debug Order

1. Check room phase.
2. Check `buildStatus` and the cached future plan.
3. Check `Memory.rooms[roomName].spawnQueue`.
4. Check harness output and private-server CPU and room snapshots.
5. Only then drill into individual roles, pathing helpers, or manager-local targeting.

15. Quick File Map

FILE	DUTY STATION
`dev/room_state.js`	Phase resolution and shared room facts.
`dev/construction_roadmap.js`	Named phase structure goals.
`dev/construction_status.js`	Shared build readiness and future-plan truth.
`dev/construction_manager.js`	Site placement.
`dev/defense_layout.js`	Terrain choke defense planning.
`dev/spawn_manager.js`	Workforce requests and recovery.
`dev/advanced_structure_manager.js`	Labs, factory, and advanced haul tasks.
`scripts/validation/solo_room_harness.js`	Deterministic solo-room validation.
`scripts/private_server/`	PTR server, upload, reset, and staging helpers.